

Università degli Studi di Milano

Corso di Laurea in
Sicurezza dei Sistemi e delle Reti Informatiche

Lezione 3 — Fibonacci: ricorsione vs. iterativo

ROBERTO ARINGHERI

Algoritmi e Strutture Dati

Modulo 7 — Unità Didattica 1

Indice

1.	INTRODUZIONE E MOTIVAZIONI	3
2.	DEFINIZIONE	3
3.	UN ALGORITMO ITERATIVO	3
4.	UN ALGORITMO RICORSIVO	3
5.	CONFRONTO	4

1. Introduzione e motivazioni

L'obiettivo di questa lezione è quello di illustrare la differenza di complessità tra un algoritmo ricorsivo ed uno iterativo prendendo come spunto la soluzione del problema di calcolare i numeri di Fibonacci.

Dopo aver ricordato la relazione ricorsiva che definisce i numeri di Fibonacci, mostreremo un algoritmo iterativo ed uno ricorsivo per la soluzione del problema. Dopo aver calcolato la complessità dei singoli algoritmi, effettueremo un confronto per determinare quale dei due algoritmi è il migliore cercando di individuarne i motivi.

2. Definizione

Denotiamo con $F(n)$ l' n -esimo numero di Fibonacci. La relazione di ricorrenza di Fibonacci è la seguente:

$$F(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ F(n-1) + F(n-2), & n \geq 2 \end{cases} \quad (1)$$

3. Un algoritmo iterativo

L'algoritmo **iterativo** fa uso di un vettore di appoggio nel quale collezionare i numeri di Fibonacci necessari al calcolo di $F(n)$.

```
long F_i( long n ) {
    long *F, i;

    F = (long *) malloc( (n+1) * sizeof(long) );
    F[0] = 0; F[1] = 1;

    for( i=2; i<=n; i++ ) F[i] = F[i-1] + F[i-2];

    return( F[n] );
}
```

la funzione del tempo di calcolo di `F_i` è la seguente:

$$T(n) = 3(n-2) + 3,$$

visto che ad ogni iterazione del ciclo, che sonon $- 2$, si eseguono 4 operazioni elementari (somma degli `F[i]`, assegnamento della somma a `F[i]`, l'incremento di `i` ed il confronto `i<=n`). Fuori dal ciclo invece eseguiamo 3 operazioni elementari, ovvero l'inizializzazione delle prime due posizioni di `F[i]` ed il `return` che può essere visto come un assegnamento. Di conseguenza, $O(T(n)) = O(n)$.

4. Un algoritmo ricorsivo

La procedura ricorsiva si basa direttamente sulla relazione descritta nell'equazione (1).

```
long F_r( long n ) {
    if ( n<=1 )
        return( (n==0) ? 0 : 1 );
    else
        return( F_r(n-1) + F_r(n-2) );
}
```

La relazione di ricorrenza e' quindi la seguente:

$$T(n) = \begin{cases} d, & n \leq 1 \\ T(n-1) + T(n-2) + c, & n > 1 \end{cases} \quad (2)$$

Applichiamo il teorema delle ricorrenze lineari di ordine costante. Visto che $\alpha = 2$ e $\beta = 0$, otteniamo che $O(T(n)) = O(2^n)$.

5. Confronto

Come si può notare, l'algoritmo iterativo risulta essere di complessità **lineare** mentre quello ricorsivo risulta **esponenziale**.

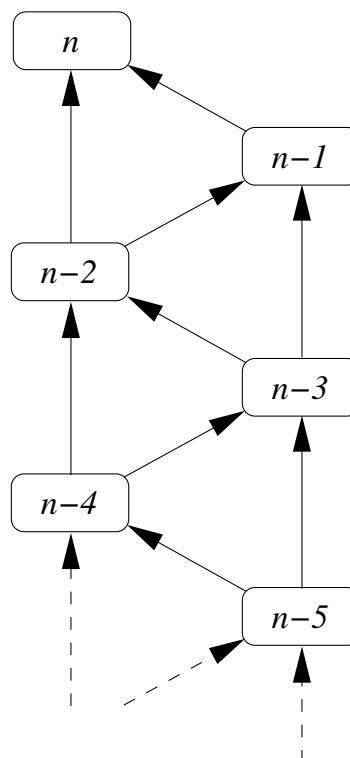


Figura 1: Numeri di Fibonacci

Una spiegazione di questo comportamento può essere data nel seguente modo.

Il grafo in figura 1 rappresenta le relazioni tra i vari numeri di Fibonacci. Ad esempio, sommando $n-1$ ed $n-2$ ottengo l' n -esimo numero di Fibonacci.

Generalizzando, il grafo di Fibonacci è un grafo $G = (N, A)$ tale che $N = \{0, \dots, n\}$ ed $A = \bigcup_{u \in N \setminus \{0,1\}} A(u)$ dove $A(u) = \{(u-2, u), (u-1, u)\}$.

L'algoritmo iterativo equivale a percorrere **un** cammino che parte dal nodo 2 al nodo n . La lunghezza di tale cammino è evidentemente $O(n)$.

Viceversa, come si può osservare dal codice della funzione, le due chiamate ricorsive con valori $n-1$ e $n-2$ equivalgono a percorrere, partendo da un nodo, entrambi i due possibili cammini che arrivano nel nodo. Lo stesso alle chiamate successive. In pratica, la procedura ricorsiva è come se enumerasse tutti i possibili cammini del grafo. Tutti i possibili cammini sono 2^n considerato che per ogni nodo, ho due archi che rappresentano due possibili scelte.